Intro to R and RStudio

SSQL Workshop

Greg Amusu
Swarthmore College

Fall 2026

Plan for the next two hours

When	What
0:00	**Install together:** R, RStudio, tidyverse, and a Quarto check
0:20	A short tour of RStudio and Quarto, plus good habits
0:40	**Guided walkthrough:** vectors, tibbles, and pipes
1:05	**You work through the handout** while I circulate
1:50	Wrap-up: staying current and where to go next

Install first. Talk second.

We start with installation so anything that goes wrong has time to get fixed.

Part 1

Let’s get set up

1 Get the workshop files

Everything for today is in one place:

github.com/swat-ssql/intro-r-workshop

1. Open the link, click the green **Code** button, then **Download ZIP**
2. **Unzip** it somewhere you'll find it again (not inside Downloads)
3. Keep the page open: the **README** links to the R and RStudio downloads we need next

The two things you're installing

R

The **language and engine**. It does the actual computing.

Free, open source, and maintained by a worldwide community.

RStudio

The **workspace** that makes R easier to use: editor, console, plots, and help in one window.

This is the program you'll open.

Order matters: install R first, then RStudio. Both download links are in the repo's README.

2 Install R

Mac | Windows

1. Go to cran.r-project.org and click **Download R for macOS**
2. Download the installer (`.pkg`) that matches your chip: **Apple silicon** (M1, M2, M3, …) or **Intel**
   Not sure? Apple menu → About This Mac.
3. Open the `.pkg` and click through the installer
4. Open **Terminal** (⌘ + Space, type "terminal") and run:

```
xcode-select --install
```

Say yes to installing the command line tools. Already have them? You're set.

3 Install RStudio

1. Go to posit.co/download/rstudio-desktop
2. Download the installer and run it (Mac: drag RStudio into Applications)
3. Double-click `intro-r-workshop.Rproj` to open the project
4. Type this into the **Console** (bottom left) and press Enter:

```
R.version.string
#> [1] "R version 4.6.x (...)"
```

An R version means RStudio and R are talking. The Files pane should list `handout.qmd`.

4 Install the tidyverse

```
install.packages("tidyverse", repos = "https://cloud.r-project.org")
```

- Takes a few minutes. Lots of scrolling text is normal.
- If it asks *"Do you want to install from sources…?"*, answer **No**.
- You install a package **once**. You load it with `library()` **every session**:

```
library(tidyverse)
```

You should see **"Attaching core tidyverse packages"** and a list of packages.

5 Check that Quarto works

You'll need **Quarto** to turn a `.qmd` file into a finished document. **RStudio already includes it**, so there's nothing extra to download.

1. **File → New File → Quarto Document…**, then click **Create** (leave HTML selected). If it offers to install packages, say **yes**
2. Save the file, then click the blue **Render** arrow. An HTML preview should open
3. Now try a PDF: change `format: html` to `format: typst` in the header and **Render** again. Typst comes with Quarto, so nothing else to install

> ⚠ **Render says Quarto isn't found, or the PDF fails?**
>
> Install Quarto from quarto.org/docs/download. HTML always works, and that's fine for today.

Checkpoint

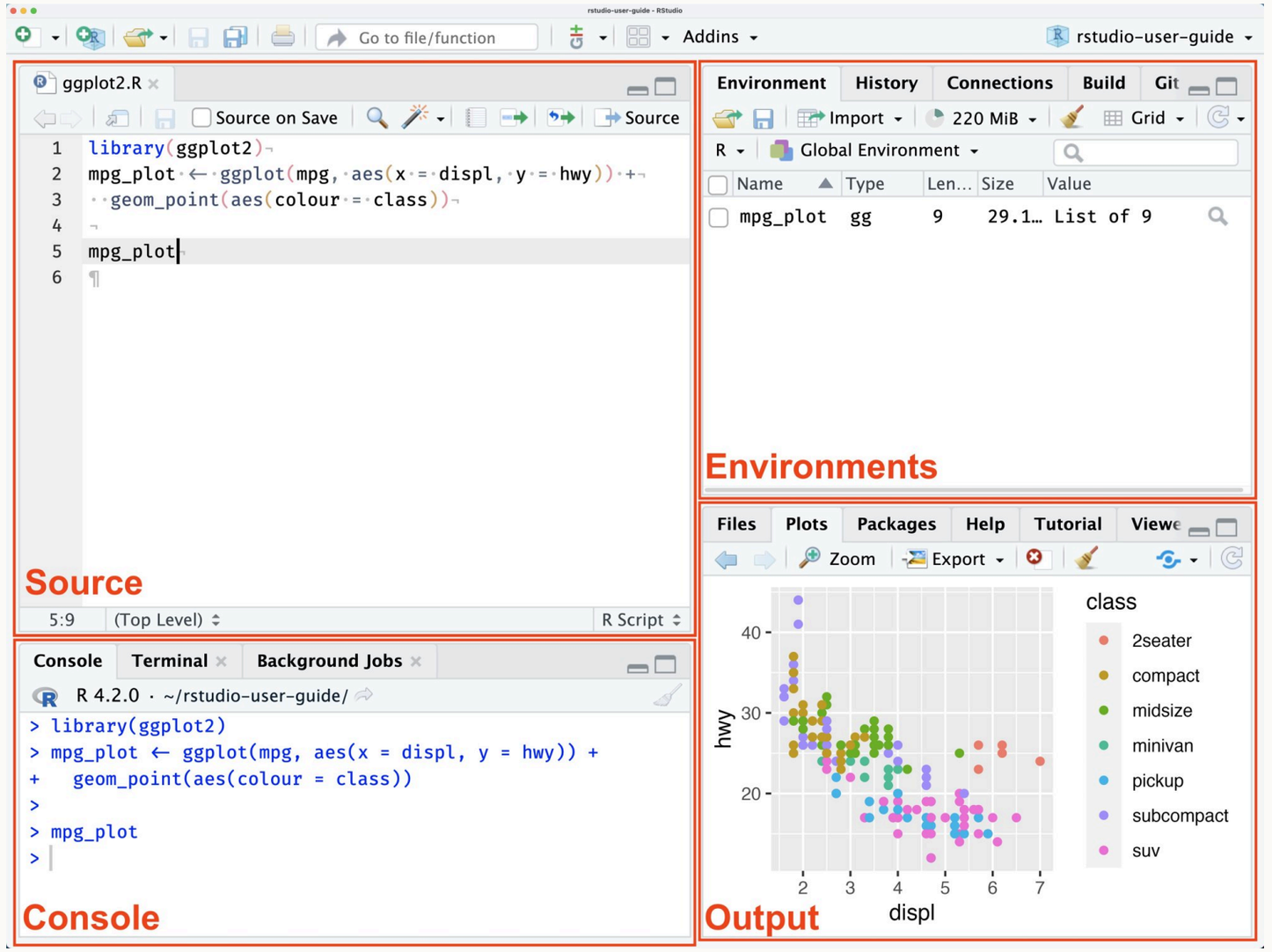
Hand up if any of these aren't green:

- [ ] The workshop project is open (the Files pane shows `handout.qmd`)
- [ ] `R.version.string` prints a version
- [ ] `library(tidyverse)` runs without an error
- [ ] A test Quarto document renders to HTML and to PDF (`format: typst`)

Part 2

A quick tour

A tour of RStudio



Where does my code live?

Console

Try things out. Quick answers.

Not saved.

Script (`.R`)

A saved list of commands.

Run a line with **⌘/Ctrl + Enter**.

Quarto (`.qmd`)

Code + writing + results in one document.

Press **Render** to build it.

Rule of thumb: if you want to keep it, it goes in a file.

Can I only use R in a Quarto file in RStudio?

No. R is the same everywhere. Only the place you type changes.

Where	Good for
R console: the Console pane, or type `R` in a terminal	quick experiments
Script (`.R`)	just code; run a line at a time or the whole file
Quarto (`.qmd`)	code + writing + results in one document
Other editors: Positron, VS Code	same R in a different workspace

```
Rscript handout_code.R     # run a whole script from a terminal
```

See the difference

`handout_code.R` in the repo holds the same code as `handout.qmd`, as a plain script. Open them side by side.

Quarto: code, writing, and results together

````
---
title: "My analysis"
format: html
---

Some **text** written in Markdown.

```{r}
mean(c(2, 4, 6))
```
````

- The top block (**YAML**) sets the title and output format: PDF, HTML, or Word
- Text goes in normal paragraphs. Code goes in **chunks**
- Press **Render** and Quarto runs the code and builds the document
- `handout.qmd` is a Quarto file, and it renders to PDF with Typst

Projects and the working directory

`getwd()` tells you which folder R is looking in right now.

- Open the workshop as a **Project**: double-click `intro-r-workshop.Rproj` (or File → Open Project)
- Now file paths are relative to the project folder:

```
read_csv("data/subset_saf.csv")
```

- Avoid `setwd("/Users/you/Desktop/...")`. It breaks on every other computer.

Objects, functions, packages

```
x <- c(2, 4, 6)         # <- stores a value in an object called x
mean(x)                 # functions look like name(inputs)
#> [1] 4
library(tidyverse)      # packages add more functions
```

- **Assign** with `<-` (shortcut: **Option/Alt + -**)
- **Call** a function with `name(inputs)`
- **Install** a package once, **load** it every session

Part 3

Good habits from day one

Stay organized

1. **One project folder** per course or analysis, with data, code, and output kept in clear places
2. Work in a **script or Quarto file**, not just the console
3. **Raw data stays raw.** Never edit the original file. Do all cleaning in code so you (or anyone else) can rerun it
4. Use clear names: `snake_case`, and `turnout` rather than `df2`. Avoid spaces in file names
5. Comment the **why**, not just the what

Know your data before you analyze it

What is one row?

A county-year? A police stop? A person? This is your **unit of observation**.

Read the codebook

What does each variable mean, in what units, and how are **missing values** coded (`NA`, `-99`, `999`)?

Look before you leap

`glimpse()`, `summary()`, `count()`. Do the ranges make sense? An age of 0 or 999 should stop you.

Remember the people

Behind each row of social data is a person. Handle it with care.

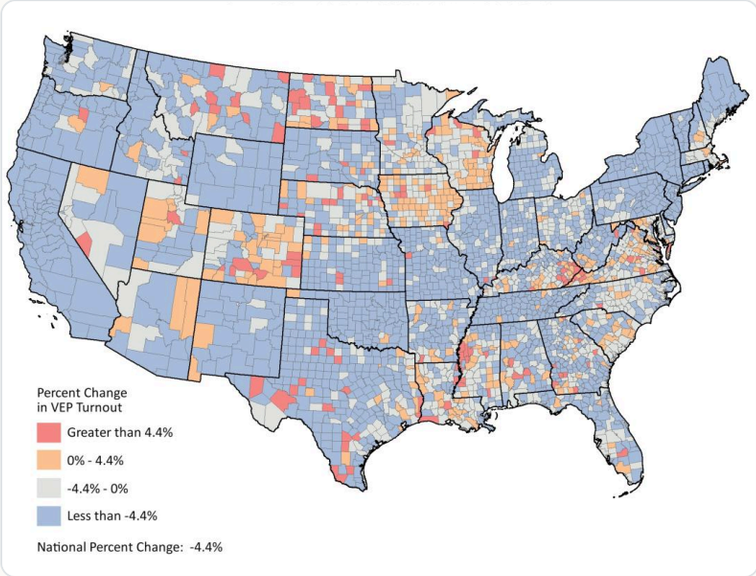
Make it reproducible

- **Restart R and run everything top to bottom** (Session → Restart R, then Render) before you submit. If it works from scratch, it works.
- **Don't save your workspace.** Tools → Global Options → General: set *Save workspace to .RData on exit* to **Never** and untick *Restore .RData into workspace at startup*
- **Know your versions:** `sessionInfo()` lists R and package versions
- Later: version control with **git and GitHub** (this workshop lives in a repo)

Part 4

Guided walkthrough: from vectors to tibbles

The data you'll meet today



Voter turnout, 2008 vs. 2012. Vectors and tibbles.



Disaster aid and the incumbent vote. Healy and Malhotra (2009).



NYC stop-and-frisk. Your questions at the end. Gelman, Fagan, and Kiss (2007).

Follow along in the handout

1. Open `handout.qmd` (in the project you opened earlier) and put your name in the header
2. Work **top to bottom**: run each chunk with the green ▶ (or **⌘/Ctrl + Shift + Enter**) and read the output
3. **Run the code as I show it.** The vectors-to-pipes code is already in the handout
4. Some chunks are blank on purpose. `handout_guide.pdf` shows the finished versions
5. Do the **Workshop Questions** on your own at the end
6. **Render** to PDF when you're done

Working with vectors

We'll take the turnout figures from Trende's chart in the handout and **combine** them into vectors, one per column.

```
actual_2008    <- c(97677100, 16564353, 11174365, 2629262, 3418041)
projected_2012 <- c(99139605, 17455952, 13134504, 3300869, 3910201)
actual_2012    <- c(93033145, 16797651, 12921270, 3230318, 2584254)
```

Math works **element by element**:

```
difference <- actual_2012 - projected_2012
difference
#> [1] -6106460  -658301  -213234   -70551 -1325947
```

Picking out elements: `[ ]`

```
actual_2008[2]
#> [1] 16564353

actual_2012[1] - projected_2012[1]
#> [1] -6106460
```

Concept check: what did each line do?

The first picks the **2nd** element. The second subtracts the **1st** projected value from the **1st** actual value.

Common vector functions

```
length(actual_2008)   # 5
mean(actual_2008)     # about 26.3 million
median(actual_2008)   # 11174365
range(actual_2008)    # 2629262 97677100
sum(actual_2008)      # 131463121
```

The handout adds `min()`, `max()`, `prod()`, `log()`, and `unique()`. Try them on your own.

Naming vectors

```
names(actual_2008)    # no names yet
#> NULL

names(actual_2008) <- c("White", "Black", "Hispanic", "Asian", "Other")
actual_2008
#>    White    Black Hispanic    Asian    Other
#> 97677100 16564353 11174365  2629262  3418041
```

```
actual_2008["White"]
#>    White
#> 97677100
```

From vectors to tibbles

A dataset is just **named vectors side by side**.

- Vectors are single columns of numbers or labels
- Real data are **tables**: many columns, each a variable
- **Tibbles** (from the tidyverse) are tables that print more clearly and play nicely with tidyverse tools
- Let’s combine our vectors into one

Building a tibble

```
turnout <- tibble(
  group = c("White", "Black", "Hispanic", "Asian", "Other"),
  actual_2008 = actual_2008,
  projected_2012 = projected_2012,
  actual_2012 = actual_2012
)
turnout
```

```
#> # A tibble: 5 × 4
#>   group    actual_2008 projected_2012 actual_2012
#>   <chr>          <dbl>          <dbl>       <dbl>
#> 1 White       97677100       99139605    93033145
#> 2 Black       16564353       17455952    16797651
#> 3 Hispanic    11174365       13134504    12921270
#> 4 Asian        2629262        3300869     3230318
#> 5 Other        3418041        3910201     2584254
```

Try breaking it

What happens if you delete `, "Other"` so that

```
group = c("White", "Black", "Hispanic", "Asian")
```

R refuses, and tells you why:

```
#> Error in `tibble()`:
#> ! Tibble columns must have compatible sizes.
#> • Size 4: Existing data.
#> • Size 5: Column `actual_2008`.
```

Columns in a table must all be the **same length**. Reading error messages is a skill.

Pipes: `%>%` and `|>`

- A pipe passes the result of one step into the next, so code reads **left to right**
- `%>%` (tidyverse) and `|>` (base R) do the same job here
- You can **chain** steps instead of nesting them

```
mean(log(x))                  # nested: read from the inside out

x %>% log() %>% mean()        # piped: read left to right
x |> log() |> mean()          # same thing, base R pipe
```

Pulling out and adding columns

`$` pulls out one column **by name**: `object$column`

```
turnout$actual_2012 - turnout$projected_2012
#> [1] -6106460  -658301  -213234   -70551 -1325947
```

`mutate()` **adds** a column to the tibble:

```
turnout <- turnout %>%
  mutate(diff_2012 = actual_2012 - projected_2012)
```

Just want the vector back? Add `%>% pull(diff_2012)`. The handout shows how.

Adding a column: the result

```
turnout
#> # A tibble: 5 × 5
#>   group    actual_2008 projected_2012 actual_2012 diff_2012
#>   <chr>          <dbl>          <dbl>       <dbl>     <dbl>
#> 1 White       97677100       99139605    93033145  -6106460
#> 2 Black       16564353       17455952    16797651   -658301
#> 3 Hispanic    11174365       13134504    12921270   -213234
#> 4 Asian        2629262        3300869     3230318    -70551
#> 5 Other        3418041        3910201     2584254   -1325947
```

The new `diff_2012` column sits at the end.

Which group had the biggest drop?

`arrange()` sorts the rows. `slice(1)` keeps the first one.

```
turnout %>% arrange(diff_2012) %>% slice(1)
```

```
#> # A tibble: 1 × 5
#>   group actual_2008 projected_2012 actual_2012 diff_2012
#>   <chr>       <dbl>          <dbl>       <dbl>     <dbl>
#> 1 White    97677100       99139605    93033145  -6106460
```

Does the answer surprise you? Why or why not? What other new columns would help here?

Part 5

Your turn

Getting unstuck

1. **Built-in help:** `?mean` or `??regression`. The examples at the bottom are the best part.
2. **Read the error message.** Common causes: a typo, a missing `)` or quote, forgetting `library()`, a wrong file path.
3. **Search the exact message** along with "R" or "tidyverse".
4. **Good references:** R for Data Science and tidyverse.org
5. **Ask.** Me, a neighbor, or your instructor.

AI chat tools

They can explain error messages, but always run what they give you and check it does what you think. Follow your course's policy.

Your turn!

Open `handout.qmd` and begin.

When you're stuck:

- Check the **Console** for an error message and read it
- Check `getwd()`: is R in the right folder?
- Ask a neighbor, then ask me

Workshop files: github.com/swat-ssql/intro-r-workshop

Part 6

Wrap-up

Keeping R, RStudio, and Quarto up to date

	How often	How to update
R	A new version each spring (4.6.0 came out April 2026), plus small fixes	Download and install the new version from CRAN, then **reinstall your packages**
RStudio	Roughly monthly. Numbered by year.month (e.g., 2026.10)	Help → Check for Updates
Quarto	Comes **bundled with RStudio**, so it updates when RStudio does	Update RStudio, or download from quarto.org
Packages	Whenever their authors update them	Tools → Check for Package Updates

Don't update in the middle of an assignment.

Update at the start of a term or a break. If everything works, you can leave it alone.

RStudio isn’t the only option: meet Positron

- A newer editor from **Posit**, the makers of RStudio, built on the same open-source base as **VS Code**
- Works with **R and Python** side by side
- Same R, same packages, same Quarto, so what you learn today carries over
- Posit says **RStudio is not going away** and will keep being maintained
- For this course we’ll stick with **RStudio**, so screenshots and instructions match

Curious? positron.posit.co

Where to go next

- Keep your project folder. You'll reuse these skills all semester
- Read: R for Data Science (2e), the first few chapters
- Try: **Positron**, if you're curious about Python or VS Code
- Keep asking questions: nobody learns R without getting stuck first

Keep learning with the SSQL

Upcoming R workshops. Register on the SSQL website.

Date	Workshop
Oct. 28	U.S. Midterm Election Modeling
Nov. 5, 6	Exploring Data in R
Nov. 9, 13	Data Visualization in R

Need help with a course or project?

Schedule an **office hours appointment** at the SSQL for course or project support.

Thank you!